

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Clase 15: HTTP (continuación)

Repaso



¿Qué es lo que escribimos en la barra de direcciones de un *browser*?

Una **URL** (*Uniform Resource Locator*) es una referencia a un recurso que especifica su ubicación en una red informática y un protocolo para recuperarlo:

`protocolo://host/carpeta1/carpeta2/.../carpetan/recurso`

Ejemplo:

`https://www.ole.com.ar/tema/lionel-messi.html`

- ▶ Protocolo: `https`
- ▶ Host: `www.ole.com.ar`
- ▶ Ubicación en el host: `tema/`
- ▶ Recurso: `lionel-messi.html`

Repaso

Es muy habitual que el resultado de una función de una [API](#) tenga formato [JSON](#). La clase `Response` tiene un método `res.json()` que, en caso de que la respuesta de la API tenga este formato, transforma su contenido en objetos Python del mismo modo que lo hacía la biblioteca [json](#).

Ejemplo:

```
import requests

def obtener_chiste():
    url = "https://official-joke-api.appspot.com/random_joke"
    response = requests.get(url)
    data = response.json()

    chiste = data["setup"] + " " + data["punchline"]
    return chiste

# Cuerpo principal del programa
chiste = obtener_chiste()
print("\nChiste recibido:")
print(chiste)
```

```
Chiste recibido:
Why don't you find hippopotamuses hiding in trees? They're really good at it.
```

Repaso

Es muy habitual que el resultado de una función de una [API](#) tenga formato [JSON](#). La clase `Response` tiene un método `res.json()` que, en caso de que la respuesta de la API tenga este formato, transforma su contenido en objetos Python del mismo modo que lo hacía la biblioteca [json](#).

Ayudamemoria:

JSON	Python
String	str
Número (entero)	int
Número (real)	float
Array	list
Booleano (true/false)	bool (True/False)
Objeto JSON	dict
null	None

HTTP: parámetros

Cuando hacemos un requerimiento HTTP se pueden incluir **parámetros** y **parámetros** (del mismo modo que al llamar a una función). Para ello, al final de la URL se incluye el signo **?** seguido de pares **clave=valor** separados por el símbolo **&**.

Ejemplo: accediendo a https://pizzaloca.com/pedido?nro_pedido=2 solicitamos el segundo pedido almacenado en el servidor:

```
{
  "tamano": "grande",
  "precio": 4300.00,
  "adicionales": ["jamón", "morrones"],
  "cliente": {
    "nombre": "Sofía",
    "telefono": "1132234532",
    "correo": "sofi04@em.com"
  }
}
```

Al hacer el requerimiento desde Python se puede usar el mismo formato en la URL:

```
response = requests.get('https://pizzaloca.com/pedido?nro_pedido=2')
```

Sin embargo...

HTTP: parámetros

Cuando son muchos los parámetros que se envían en un requerimiento HTTP, incluirlos todos en la URL resulta desprolijo.

https://pizzaloca.com/pedido?nro_pedido=2&fecha=12-03-98&cortada=True&aceitunas=False

La función `get(...)` admite un parámetro opcional `params`, para indicar en un diccionario de Python los valores de los parámetros que son enviados al servidor. En este diccionario, tanto las claves como los valores deben ser strings.

```
valores_params = dict()
valores_params['nro_pedido'] = '2'
valores_params['fecha'] = '12-03-98'
valores_params['cortada'] = 'True'
valores_params['aceitunas'] = 'False'
resp = requests.get('https://pizzaloca.com/pedido', params=valores_params)
```

HTTP: parámetros

Ejercicio 1. La API pública <https://api.open-meteo.com/v1/forecast> permite consultar el clima en una ubicación dada. Por ejemplo, puede obtenerse la temperatura actual indicando la latitud y longitud del lugar.

El *endpoint* básico es, por ejemplo:

<https://api.open-meteo.com/v1/forecast?latitude=52.52&longitude=13.41¤t.weather=true> y la respuesta tiene la siguiente forma:

```
{
  "current_weather": {
    "temperature": 14.3,
    "windspeed": 5.6
    ...
  }
  ...
}
```

Escribir un programa en Python que: (i) solicite latitud y longitud; (ii) consulte la API con la librería `requests`; y (iii) muestre por consola la temperatura actual (clave `temperature`).

Buenos Aires: lat=-34.61, lon=-58.38
Madrid: lat=40.42, lon=-3.70
Ciudad de México: lat=19.43, lon=-99.13

HTTP: parámetros

Ejercicio 2.

Usando la misma API pública de *Open-Meteo* vista en el ejercicio anterior, se pueden consultar datos históricos del clima para una ubicación determinada y un intervalo de fechas.

Los parámetros que deben incluirse en la consulta son los siguientes:

- ▶ `latitude`: latitud de la ubicación
- ▶ `longitude`: longitud de la ubicación
- ▶ `start_date`: fecha de inicio (YYYY-MM-DD)
- ▶ `end_date`: fecha de fin (YYYY-MM-DD)
- ▶ `hourly`: variable meteorológica a recuperar.

La variable `hourly` puede tomar valores como `temperature_2m` (temperatura del aire a 2 metros del suelo, en °C), `precipitation` (precipitación en mm), `windspeed_10m` (velocidad del viento a 10 metros, en km/h), entre otros.

En este ejercicio se trabajará con la variable `temperature_2m`.

(Continúa)

HTTP: parámetros

(Continuación)

La API devuelve un JSON que incluye, entre otros, un campo "hourly" con la lista "temperature_2m", que contiene las temperaturas registradas hora a hora en el período solicitado.

```
{
  ...
  "hourly": {
    "temperature_2m": [
      15.2,
      14.9,
      14.3,
      ...
    ]
  }
}
```

Se pide escribir un programa en Python que:

1. solicite al usuario la latitud, la longitud, la fecha de inicio y la fecha de fin;
2. utilice la librería `requests` para consultar la API de *Open-Meteo* con esos parámetros;
3. almacene la respuesta en un diccionario usando `.json()` y muestre por consola la temperatura máxima registrada e el período indicado.

Repaso de la clase de hoy

- ▶ Consultas HTTP con parámetros

Con lo visto ya pueden completar la sección 2 de la guía de Métodos Analíticos y Programación.